

DART XML 원문 구조 조사

항목	내용
작업 단계	04
작업 영역	공시 원문 구조 조사
문서 버전	v2.0
최종 갱신일	2026-08-11
조사 대상	<code>disclosure_documents</code> 에 등록된 <code>DART_XML</code> 문서
조사 수량	3,147개
최종 상태	완료 (<code>SUCCESS</code> 3,147건, 실패 0건)
최종 리포트	<code>xml-additional-structure-page-000.csv ~ 006.csv</code>

1. 문서 목적

이 문서는 대회 제공 DART XML 원문 3,147개의 구조를 전수 조사한 결과와, 그 결과로 확인한 원문 파서 설계 방향을 기록한다.

이번 조사는 다음 질문에 답하기 위해 수행했다.

- 모든 XML을 같은 진입점에서 읽을 수 있는가?
- 장·절 구조는 몇 단계까지 존재하는가?
- 제목과 문단은 어떤 위치에 나타나는가?
- 표가 얼마나 많고 중첩 표가 실제로 존재하는가?
- 이미지, 줄바꿈, 페이지 구분, 주식 관련 태그가 존재하는가?
- 표준 XML 파서가 읽지 못하는 문법 오류는 어떤 유형인가?
- 대형 XML을 스트리밍 방식으로 안정적으로 처리할 수 있는가?
- 현재 작성 중인 파서 모델과 실제 데이터가 충돌하는 부분은 무엇인가?

구조 조사의 목적은 공시의 의미를 추출하는 것이 아니다. 실제 원문 파서를 구현하기 전에 **데이터에 존재하는 구조와 예외를 확인하고 파서 규칙의 근거를 만드는 것**이 목적이다.

2. 조사 범위

2.1 포함 범위

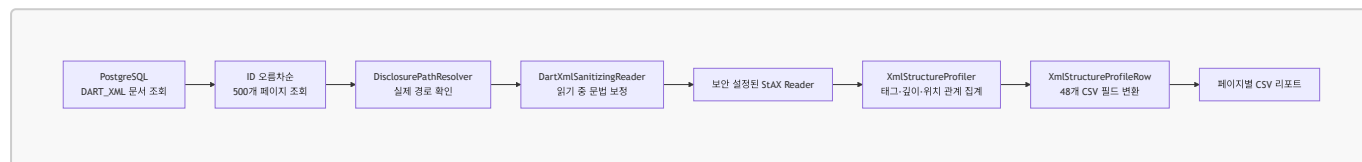
- PostgreSQL `disclosure_documents` 테이블
- `content_format` = `DART_XML`
- `document_role` = `MAIN` 또는 `ATTACHMENT`
- `source_group` = `periodic`, `major`, `holding`
- 페이지당 최대 500개
- 총 7페이지

2.2 제외 범위

- HTML 원문 1,472개
- PDF 원문 3개
- 공시 내용의 의미 분석
- 금액·날짜·비율 등 핵심 사실 추출
- 검색 청크 생성 및 DB 적재
- LLM 답변 생성

HTML과 PDF는 DART XML과 읽기 방식이 다르므로 별도 파서와 별도 구조 조사가 필요하다.

3. 조사 방식



3.1 스트리밍 처리

XML 전체를 메모리에 올리는 DOM 방식 대신 StAX를 사용했다. 태그의 시작, 문자, 종료 이벤트를 앞에서부터 한 번씩 읽으며 집계한다.

이 방식은 최대 약 32MB인 XML도 전체 트리를 메모리에 만들지 않고 처리할 수 있다.

3.2 원본 보존

문법 보정은 `DartXmlSanitizingReader`가 원문을 읽어 파서에 전달하는 순간에만 적용한다.

- 원본 XML 파일은 수정하지 않는다.
- DB 경로와 메타데이터도 수정하지 않는다.
- 어떤 보정이 몇 번 적용됐는지 CSV에 기록한다.
- 구조 조사기와 실제 파서가 같은 보정 Reader를 사용한다.

3.3 실패 격리

파일 하나가 실패하더라도 전체 페이지를 중단하지 않고 다음 파일을 계속 조사하도록 구성했다. 실패 행에는 최하위 예외 종류, 메시지, XML 행과 열을 기록한다.

최종 조사에서는 실패가 0건이었다.

4. 완료 기준과 검증 결과

4.1 완료 기준

- DB의 DART XML 전체 건수와 CSV 전체 행 수가 일치한다.
- 모든 `disclosure_document_id`가 고유하다.
- `source_doc_id + file_name` 조합이 고유하다.
- 각 페이지 내부 ID가 오름차순이다.
- 다음 페이지 시작 ID가 이전 페이지 마지막 ID보다 크다.
- 7개 CSV가 같은 48개 열을 가진다.
- 성공 행의 태그별 합계가 `total_element_count`와 일치한다.
- SECTION 단계별 요약과 개별 집계 필드가 일치한다.

- 표 개수, 중첩 표 개수, 최대 표 깊이가 논리적으로 모순되지 않는다.
- 문단·제목의 내부 위치 집계가 전체 개수를 넘지 않는다.

4.2 최종 검증

검증 항목	결과
DB의 DART XML 문서 수	3,147
최종 CSV 행 수	3,147
고유 <code>disclosure_document_id</code>	3,147
고유 <code>source_doc_id + file_name</code>	3,147
중복 문서	0
누락된 페이지 구간	0
성공	3,147
실패	0
CSV 열 수	48
열 구조가 다른 CSV	0
숫자 변환 오류	0
태그 합계 불일치	0
SECTION 집계 불일치	0
표 깊이·중첩 관계 오류	0
필수 식별 정보 누락	0

`XmlStructureProfilerTest`도 통과했다. 전체 `gradlew test`에서는

`BackendApplicationTests.contextLoads()`가 테스트 환경의 PostgreSQL 연결 실패로 실패했지만, 구조 조사기 단위 테스트 및 CSV 결과와는 별개의 테스트 인프라 문제다.

5. 페이지별 결과

페이지	리포트	대상	성공	실패	파일별 소요시간 합계
0	<code>xml-additional-structure-page-000.csv</code>	500	500	0	221,259ms
1	<code>xml-additional-structure-page-001.csv</code>	500	500	0	181,144ms
2	<code>xml-additional-structure-page-002.csv</code>	500	500	0	234,347ms
3	<code>xml-additional-structure-page-003.csv</code>	500	500	0	227,965ms
4	<code>xml-additional-structure-page-004.csv</code>	500	500	0	233,925ms
5	<code>xml-additional-structure-page-005.csv</code>	500	500	0	178,965ms
6	<code>xml-additional-structure-page-006.csv</code>	147	147	0	54,677ms

페이지	리포트	대상	성공	실패	파일별 소요시간	합계
합계	7개 리포트	3,147	3,147	0		1,332,282ms

파일별 소요시간 합계는 각 행의 `elapsed_millis`를 합한 값이다. 애플리케이션 시작, DB 조회, CSV 저장, 컨테이너 시작 시간은 포함하지 않는다.

소규모 검증용 파일과 이전 필드 버전 리포트는 최종 집계에서 제외했다.

```
xml-additional-structure-test-050.csv
xml-structure-profile.csv
xml-structure-profile-200.csv
xml-structure-profile-page-000.csv ~ 006.csv
```

6. 데이터 구성

6.1 공시 그룹

source_group	문서 수
periodic	1,466
holding	1,083
major	598
합계	3,147

6.2 문서 역할

document_role	문서 수
MAIN	2,732
ATTACHMENT	415
합계	3,147

주 문서뿐 아니라 첨부 문서도 독립적인 XML 구조와 내용을 가진다. 실제 파서와 검색 데이터에는 문서 역할을 보존해야 한다.

7. 공통 XML 구조

7.1 공통 진입점

항목	결과
루트가 <code>DOCUMENT</code> 인 문서	3,147
<code>DOCUMENT-NAME</code> 태그	3,147
<code>document_name</code> 추출 성공	3,147

항목	결과
BODY 태그	3,147

전체 XML이 DOCUMENT를 루트로 사용하므로 실제 파서는 DOCUMENT를 공통 진입점으로 사용할 수 있다.

대표적으로 관찰되는 구조 요소는 다음과 같다. 모든 문서가 아래 하위 요소를 전부 갖는다는 뜻은 아니다.

```

DOCUMENT
├── DOCUMENT-NAME
├── COVER
├── BODY
│   ├── SECTION-1
│   │   ├── TITLE
│   │   ├── P
│   │   ├── TABLE
│   │   └── SECTION-2
│   │       ├── SECTION-3
│   │       └── SECTION-4
│   └── ...
├── LIBRARY
└── EXTRACTION

```

7.2 문서 복잡도

항목	결과
전체 시작 태그 수	66,366,805
문서당 평균 시작 태그 수	약 21,089
XML 최대 깊이 중앙값	11
XML 최대 깊이 최댓값	19
문서별 고유 태그 수 최솟값	18
문서별 고유 태그 수 중앙값	30
문서별 고유 태그 수 최댓값	33

파일마다 길이는 크게 다르지만, 태그 종류 자체는 비교적 제한적이다. 따라서 태그별 처리 규칙을 명시적으로 관리하는 방식이 가능하다.

8. SECTION 계층 조사

8.1 태그 개수

태그	전체 개수
SECTION-1	21,405

태그	전체 개수
SECTION-2	52,756
SECTION-3	11,690
SECTION-4	492
SECTION-5 이상	0

8.2 문서별 최대 SECTION 단계

최대 단계	문서 수	비율
1	584	18.56%
2	415	13.19%
3	1,716	54.53%
4	432	13.73%
합계	3,147	100%

8.3 파서 결론

- SECTION-4는 예외적인 한두 파일이 아니라 432개 문서에서 발견됐다.
- 파서는 SECTION-1, SECTION-2, SECTION-3을 하드코딩하면 안 된다.
- ^SECTION-(\d+)\$ 형태로 단계를 해석해야 한다.
- 현재 데이터의 최대값은 4지만 결과 모델은 향후 SECTION-5 이상도 수용할 수 있어야 한다.
- 섹션에는 단계, 문서 내 순서, 제목, 부모 섹션, 원문 시작·끝 위치를 보존해야 한다.

9. 제목과 문단

항목	개수
TITLE	147,353
P	8,382,491
TITLE 내부 P	0
TABLE 내부 P	7,137,294
TABLE 내부 TITLE	0

전체 문단의 약 **85.15%**가 표 내부에 있다.

이 결과로 다음을 확정한다.

- 현재 데이터에서는 TITLE 안의 P를 중복 수집하는 사례가 발견되지 않았다.
- TITLE과 일반 P는 별도 블록으로 구분할 수 있다.
- 문단의 대부분이 표 내부에 있으므로 표 내부 P를 일반 문단으로 별도 생성하면 중복이 매우 많이 발생한다.
- 표 내부 텍스트는 표 파서가 책임져야 한다.

- 일반 문단의 공백 정규화와 표 셀의 줄바꿈 정규화는 서로 다른 규칙을 사용해야 한다.

10. 표 구조

10.1 전체 집계

항목	개수
TABLE	1,577,395
TR	8,054,877
TH	4,051,740
TD	19,141,038
중첩 TABLE	18,165
중첩 표가 있는 문서	662
최대 표 깊이	3

문서당 평균 표 개수는 약 501개이며, 전체 문서의 약 21.04%에 중첩 표가 존재한다.

10.2 문서별 최대 표 깊이

최대 표 깊이	문서 수
1	2,485
2	660
3	2

최대 깊이 3인 대표 문서는 다음과 같다.

접수번호	그룹	파일명	표 수	중첩 표 수
20250515002264	periodic	20250515002264.xml	1,689	21
20250814002734	periodic	20250814002734.xml	2,219	41

10.3 파서 결론

표를 하나의 문자열로 합치는 방식은 최초 프로토타입 확인에는 사용할 수 있지만 최종 구조로는 부족하다.

최종 파싱 모델은 최소한 다음 관계를 보존해야 한다.

```
ParsedDisclosureTable
└─ rows: List<ParsedDisclosureTableRow>
    └─ cells: List<ParsedDisclosureTableCell>
        ├── cellType: HEADER | DATA
        ├── rowSpan
        └── colSpan
```

```
└─ text
└─ nestedTables
```

필요한 처리 원칙은 다음과 같다.

- **TR** 경계를 보존한다.
- **TH**와 **TD**를 구분한다.
- 셀의 **rowspan**, **colspan** 속성이 있다면 보존한다.
- 셀 안의 **p**와 줄바꿈을 유지한다.
- 내부 **TABLE**을 바깥 표와 분리된 하위 표로 표현한다.
- 검색용 텍스트 직렬화는 구조 파싱 이후 별도 변환기가 수행한다.

11. 이미지·페이지 구분·주석 후보

11.1 이미지 후보

태그	개수
IMAGE	4,737
IMG	4,737
IMG-CAPTION	4,737
이미지 후보가 있는 문서	1,789

세 태그의 전체 개수가 같다는 점은 반복되는 이미지 묶음 구조가 있음을 시사한다. 하지만 집계만으로 세 태그가 항상 하나의 논리적 이미지를 구성한다고 확정할 수는 없다.

대표 원문을 확인한 뒤 다음 정보를 파싱할지 결정해야 한다.

- 이미지 파일 또는 식별자
- 캡션
- 대체 텍스트
- 이미지가 속한 섹션과 순서
- 검색 본문 포함 여부

11.2 줄바꿈과 페이지 구분

태그	개수
BR	0
LINEBREAK	0
LINE-BREAK	0
PGBRK	118,176

명시적인 줄바꿈 후보 태그는 없었지만 **PGBRK**는 많이 존재한다.

PGBRK를 단순 공백으로 없앨지, 원문 위치를 위한 페이지 경계로 보존할지 정책을 정해야 한다. 또한 XML 문자 이벤트에 포함된 실제 개행은 이 태그 집계와 별개이므로, 표 셀과 일반 문단의 개행 보존 규칙도 별도로 필요하다.

11.3 주석과 각주 후보

항목	결과
NOTE , FOOTNOTE , FOOT-NOTE , ANNOTATION 후보	0
XML 문법 주석	0

이 결과는 현재 후보 이름과 일치하는 태그가 없었다는 뜻이다. 공시의 의미상 주석이나 각주가 없다는 뜻은 아니다. 표의 특정 행, 일반 문단, 기호 표현으로 들어 있을 수 있으므로 의미 기반 각주 처리는 본문 파싱 결과를 추가로 확인해야 한다.

12. 주요 태그 분포

태그	개수	파서에서의 의미
DOCUMENT	3,147	문서 루트
DOCUMENT-NAME	3,147	문서명
BODY	3,147	본문 영역
TITLE	147,353	섹션 제목 또는 본문 소제목
P	8,382,491	일반 문단 또는 표 내부 문단
TABLE	1,577,395	표 시작
TR	8,054,877	표 행
TH	4,051,740	표 헤더 셀
TD	19,141,038	표 일반 셀
THEAD	534,959	표 헤더 영역
TBODY	1,577,402	표 본문 영역
COLGROUP	1,577,395	표 열 그룹
COL	4,884,430	표 열 정의
PGBRK	118,176	페이지 경계 후보
IMAGE	4,737	이미지 컨테이너 후보
IMG	4,737	이미지 본체 후보
IMG-CAPTION	4,737	이미지 캡션 후보

13. XML 문법 보정 결과

보정 유형	발생 문서	문서 비율	전체 보정 횟수
단독 &	1,620	51.48%	84,075
비구조적 <	1,352	42.96%	42,699
잘못된 속성 따옴표	79	2.51%	662

보정이 필요한 문서가 많으므로 `DartXmlSanitizingReader`는 임시 우회 코드가 아니라 실제 파서 경로의 필수 구성 요소다.

확인된 보정 유형은 다음과 같다.

유형	원인 예시	처리 방식
단독 &	본문에 XML 엔티티가 아닌 & 사용	유효한 엔티티가 아닌 경우에만 &로 전달
일반 비교식의 <	본문에 비교 기호로 사용	실제 태그 시작이 아니면 <로 전달
속성 내부 따옴표	ENG="Small office ("SOHO")" 형태	내부 따옴표를 "로 전달
속성 시작 직후 중복 따옴표	ENG=" "Snow Corporation" 형태	시작 직후 중복 따옴표 제거
태그처럼 작성된 본문 표현	<CG>, <MANIFESTO> 등	확인된 이름만 일반 텍스트로 이스케이프

현재 확인된 비구조적 꺾쇠 표현 이름은 다음과 같다.

```
STS, CG, BGMI, DREAM, MANIFESTO,
DATABADA, GRANDATA, SIT, IIT, SHEESH
```

모든 <문자열>을 일괄 이스케이프하지 않는다. 그렇게 하면 정상 XML 태그도 본문으로 바뀔 수 있기 때문이다.

실제 파서 통합 테스트에서는 보정 후 파싱 성공 여부뿐 아니라, 보정된 문자가 결과 텍스트에서 원래 의미대로 복원되는지도 검증해야 한다.

14. XML 보안 설정

`DartXmlInputFactoryProvider`는 다음 설정을 적용한다.

- DTD 비활성화
- 외부 엔티티 비활성화
- 외부 파일 및 URL 해석 차단
- 네임스페이스 인식 활성화
- 연속 문자 이벤트 병합 활성화
- 호출마다 별도 `XMLInputFactory` 생성

구조 조사기와 실제 파서가 같은 Provider를 사용하므로, 구조 조사에서 검증한 보안 설정이 실제 파싱 경로에도 적용된다.

15. 파일 크기와 성능

15.1 파일 크기

항목	결과
전체 파일 크기	5,531,987,496 bytes
평균 파일 크기	약 1,757,861 bytes
중앙 파일 크기	371,876 bytes
95백분위 파일 크기	7,152,660 bytes
최대 파일 크기	32,199,881 bytes

15.2 조사 시간

항목	결과
파일별 시간 합계	1,332,282ms, 약 22분 12초
파일당 평균	423.35ms
파일당 중앙값	187ms
파일당 95백분위	1,447ms
최대	5,614ms

가장 오래 걸린 파일은 다음과 같다.

접수번호	그룹	역할	크기	조사 시간
20260311004614	periodic	MAIN	32,172,847 bytes	5,614ms

대형 파일도 수 초 안에 처리됐으므로 일회성 내부 데이터 준비 프로그램으로서는 충분하다. 현재는 병렬 처리나 복잡한 성능 최적화보다 파싱 결과의 정확성과 구조 보존이 우선이다.

16. CSV 결과 필드

최종 추가 구조 조사 리포트는 48개 열을 가진다.

16.1 문서 식별과 메타데이터

```
disclosure_document_id, source_doc_id, receipt_no,
source_group, raw_subtype, report_name, correction,
file_name, document_role, content_format,
file_size_bytes, relative_path
```

16.2 기본 XML 구조

```
root_element_name, document_name,
max_depth, distinct_tag_count, total_element_count,
tag_counts_summary
```

16.3 SECTION·제목·문단·표 구조

```
section_1_count, section_2_count, section_3_count,
max_section_level, section_4_plus_count,
section_level_counts_summary,
title_count, paragraph_count,
paragraph_inside_title_count,
table_count, table_row_count,
table_header_count, table_cell_count,
nested_table_count, max_table_depth,
paragraph_inside_table_count,
title_inside_table_count
```

16.4 특수 구조·보정·오류

```
line_break_tag_count, xml_comment_count,
image_candidate_tag_counts,
note_candidate_tag_counts,
repaired_ampersand_count,
repaired_less_than_count,
repaired_attribute_quote_count,
elapsed_millis, status,
error_type, error_line, error_column, error_message
```

17. 현재 파서와 조사 결과의 충돌

현재 `DartXmlDisclosureParser`는 구조를 이해하기 위한 첫 번째 프로토타입이다. 전수 조사 결과를 기준으로 다음 부분은 전체 파싱 전에 반드시 수정해야 한다.

17.1 SECTION-4 미지원

현재 파서의 `isSectionTag()`는 `SECTION-1`부터 `SECTION-3`까지만 허용하고, `ParsedDisclosureSection`도 `level`을 1~3으로 제한한다.

하지만 실제로는 432개 문서에 `SECTION-4`가 존재한다.

따라서 다음 두 부분을 함께 수정해야 한다.

- `DartXmlDisclosureParser.isSectionTag()`를 정규식 기반 `SECTION-N` 판별로 변경
- `ParsedDisclosureSection`의 `level <= 3` 제한 제거

17.2 표 평탄화

현재 파서는 표를 행·셀 객체로 만들지 않고 `StringBuilder` 하나에 합친다.

다음 이유로 최종 방식으로 사용할 수 없다.

- `TH`와 `TD`의 의미가 사라진다.
- `rowspan`, `colspan`을 보존하지 못한다.
- 중첩 표 18,165개를 구분하지 못한다.
- 표 내부 문단이 전체 문단의 85.15%이므로 검색·수치 추출 품질에 큰 영향을 준다.

17.3 줄바꿈 손실

현재 `normalizeText()`는 모든 연속 공백과 개행을 하나의 공백으로 바꾼다.

일반 문단에는 사용할 수 있지만 표 셀, 여러 줄 주석, 페이지 경계에는 별도 정규화 규칙이 필요하다.

17.4 이미지와 페이지 경계 미처리

현재 파서는 `IMAGE`, `IMG`, `IMG-CAPTION`, `PGBRK`를 별도 결과로 보존하지 않는다.

이미지 후보가 1,789개 문서에 존재하고 `PGBRK`가 118,176회 있으므로, 무시 여부도 명시적인 정책으로 결정해야 한다.

17.5 파서 통합 테스트 부족

현재 구조 조사기 테스트는 있지만 `DartXmlDisclosureParser`의 실제 대표 원문 통합 테스트는 없다.

전수 파싱 전에 공시 그룹, 문서 역할, SECTION-4, 중첩 표, 이미지, 보정 유형을 포함한 fixture 테스트가 필요하다.

18. 확정된 파서 설계 원칙

1. XML 입력 검증, 보정 Reader, 보안 InputFactory는 구조 조사기와 실제 파서가 공유한다.
2. `DOCUMENT`를 공통 진입점으로 사용한다.
3. `SECTION-N`을 동적으로 해석하고 부모·자식 계층을 유지한다.
4. 문서·섹션·블록에는 원문 순서와 시작·끝 행을 보존한다.
5. 제목, 문단, 표, 이미지 등 블록 유형을 구분한다.
6. 표는 행·셀·중첩 표 구조로 파싱한다.
7. 표 안의 `P`는 일반 문단으로 중복 생성하지 않는다.
8. 일반 문단 정규화와 표 셀 정규화를 분리한다.
9. 구조 파싱과 검색 청크 생성을 별도 클래스로 분리한다.
10. 구조화 결과가 확정된 후 DB 엔티티와 테이블을 설계한다.
11. 보정 횟수와 파싱 상태를 문서 단위로 추적한다.
12. `MAIN`과 `ATTACHMENT`를 모두 처리하면서 역할을 보존한다.

19. 조사 결과만으로 확정할 수 없는 것

- 태그 개수만으로 공시의 의미상 중요도를 판단할 수 없다.
- 이미지 후보 태그 세 개가 항상 하나의 이미지 묶음인지는 대표 원문 확인이 필요하다.
- 주석 후보 태그가 0이라고 해서 의미상 각주가 없는 것은 아니다.
- 줄바꿈 후보 태그가 0이라고 해서 원문 문자 데이터에 개행이 없는 것은 아니다.
- 표의 실제 `rowspan`, `colspan` 사용 방식은 속성 조사 또는 실제 표 파싱 과정에서 확인해야 한다.

- 구조 조사 성공은 현재 작성된 `DartXmlDisclosureParser`가 3,147개를 모두 올바르게 구조화한다는 뜻이 아니다.
- 이번 결과는 HTML과 PDF 파싱 가능성을 보장하지 않는다.

20. 다음 구현 순서

1. `SECTION-N` 일반화
 - `DartXmlDisclosureParser`
 - `ParsedDisclosureSection`
2. 표 결과 모델 정의
 - `ParsedDisclosureTable`
 - `ParsedDisclosureTableRow`
 - `ParsedDisclosureTableCell`
3. 표 행·셀·중첩 표 파서 구현
4. 일반 문단과 표 셀의 공백·개행 정규화 분리
5. `PGBRK`와 이미지 처리 정책 확정
6. 대표 원문 통합 테스트 작성
 - `periodic, major, holding`
 - `MAIN, ATTACHMENT`
 - `SECTION-4`
 - 표 깊이 3인 두 문서
 - 세 가지 XML 보정 유형
 - 이미지 후보 포함 문서
7. 소규모 실제 파싱 배치 실행 및 결과 검수
8. 전체 3,147개 파싱 성공 여부와 결과 품질 검증
9. 결과 모델이 안정화된 뒤 DB 스키마와 적재 배치 구현
10. 검색 청크와 공시 근거 연결 구현

21. 실행 설정

조사 중에는 페이지와 파일명을 함께 변경했다.

```
FOLIOLENS_PROFILE_XML_BATCH_ON_STARTUP=true
FOLIOLENS_PROFILE_XML_BATCH_PAGE=0
FOLIOLENS_PROFILE_XML_BATCH_LIMIT=500
FOLIOLENS_PROFILE_XML_BATCH_REPORT_FILE_NAME=xml-additional-structure-page-000.csv
FOLIOLENS_PROFILE_REPORT_DIRECTORY=./reports
```

전체 조사가 끝났으므로 현재는 자동 재실행을 비활성화한다.

```
FOLIOLENS_PROFILE_XML_BATCH_ON_STARTUP=false
```

페이지 방식은 `disclosure_document_id` 오름차순과 offset 기반이므로 조사 도중 DB를 재적재하거나 ID를 변경하면 페이지가 겹치거나 누락될 수 있다. 이번 최종 결과는 모든 페이지 경계와 고유 ID를 다시 검증했다.

22. 관련 코드

XML 공통 입력

```
backend/src/main/java/com/foiolens/backend/disclosure/infrastructure/xml/  
├ DartXmlInputFactoryProvider.java  
├ DartXmlSanitizingReader.java  
└ DartXmlSourceFileValidator.java
```

구조 조사

```
backend/src/main/java/com/foiolens/backend/disclosure/infrastructure/profiling/  
├ XmlAdditionalStructureProfile.java  
├ XmlStructureProfile.java  
├ XmlStructureProfiler.java  
├ XmlStructureProfileStatus.java  
├ XmlStructureProfileRow.java  
├ XmlStructureProfileBatchResult.java  
├ XmlStructureProfileBatchService.java  
├ XmlStructureProfileReportWriter.java  
└ XmlStructureProfileBatchRunner.java
```

현재 파서 프로토타입

```
backend/src/main/java/com/foiolens/backend/disclosure/infrastructure/parsing/  
├ DartXmlDisclosureParser.java  
├ ParsedDisclosureDocument.java  
├ ParsedDisclosureSection.java  
├ ParsedDisclosureBlock.java  
└ ParsedDisclosureBlockType.java
```

테스트

```
backend/src/test/java/com/foiolens/backend/disclosure/infrastructure/profiling/  
└ XmlStructureProfilerTest.java
```

23. 최종 리포트

```
reports/  
├ xml-additional-structure-page-000.csv  
├ xml-additional-structure-page-001.csv  
├ xml-additional-structure-page-002.csv  
└ xml-additional-structure-page-003.csv
```

```
| xml-additional-structure-page-004.csv  
| xml-additional-structure-page-005.csv  
| xml-additional-structure-page-006.csv
```

24. 완료 체크리스트

- ☒ XML 공통 보정 Reader 분리
- ☒ XML 보안 InputFactory 분리
- ☒ 단일 파일 구조 조사기 구현
- ☒ SECTION 단계와 SECTION-4 이상 조사
- ☒ 표 중첩과 최대 깊이 조사
- ☒ 제목·문단·표 내부 위치 관계 조사
- ☒ 이미지·주석·줄바꿈 후보 조사
- ☒ 속성 따옴표 보정 횟수 기록
- ☒ 페이지 단위 배치 구현
- ☒ DART XML 3,147개 전수 조사
- ☒ 중복·누락·페이지 경계 검증
- ☒ 최종 실패 0건 확인
- ☒ 조사 결과 문서화
- ☐ SECTION-N 파서 일반화
- ☐ 표 행·셀·중첩 구조 파서 구현
- ☐ 이미지와 PGBRK 처리 정책 확정
- ☐ 실제 원문 파서 통합 테스트
- ☐ 전체 XML 구조화 파싱 검증
- ☐ 구조화 결과 DB 적재
- ☐ 검색 청크 및 근거 연결 구현